# Advancing Industry 4.0 Control Systems with MOM-based Hybrid Architecture

Rupali Rastogi*
Central Research Laboratory
Bharat Electronics Ltd.
Ghaziabad, IN-UP, India
rupalirastoginow@gmail.com

Ajay Arya
Central Research Laboratory
Bharat Electronics Ltd.
Ghaziabad, IN-UP, India
ajayaryanow@gmail.com

Pushpinder Kaur Josan
Central Research Laboratory
Bharat Electronics Ltd.
Ghaziabad, IN-UP, India
pushpinderkaurjosan@bel.co.in

Debasis Biswas
Central Research Laboratory
Bharat Electronics Ltd.
Ghaziabad, IN-UP, India
debasisbiswass@bel.co.in

## Abstract

Efficiently managing the increasing complexity of Industry 4.0 system's implementation requires advanced system architectures that can handle high volumes of data and provide rapid responses. Architecture approach plays a critical role in determining the performance and scalability of software systems. This research paper explores and compares three architectural approaches—Monolithic, Microservice, and Hybrid—in the context of a Traffic Management System (TMS), which is a type of Industry 4.0 Supervisory Control and Data Acquisition (SCADA) system. Traditional monolithic architecture (MA) often struggles with scalability, flexibility, and performance. The microservice architecture (MSA) addresses these issues by breaking down the system into smaller, independent services, each handling a specific function. However, it introduces complexity in communication and data consistency. The proposed hybrid architecture (HA) leverages the strengths of both approaches by combining modular microservices with a shared, centralized database, and employs message-oriented middleware (MOM) protocol RabbitMQ for inter-service communication. A case study implementation of the TMS is conducted to evaluate the performance of these architectures with key performance metrics, including CPU utilization, throughput, and average error rate. Through this study, this paper explores that the HA offers a balanced solution, providing the scalability and resilience of microservices with the simplicity and centralized data management of monolithic systems.

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*ICIT 2024, Kuala Lumpur, Malaysia*
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1737-6/2024/12
https://doi.org/10.1145/3718391.3727888

## CCS Concepts

• **Software and its engineering**; • **Software organization and properties**; • **Software system structures**; • **Software architectures**; • **Publish-subscribe / event-based architectures**;

## Keywords

Microservice Architecture, Migration to Microservices, Monolithic to Microservices, SCADA, MOM protocol, System Performance

**ACM Reference Format:**
Rupali Rastogi, Ajay Arya, Pushpinder Kaur Josan, and Debasis Biswas. 2024. Advancing Industry 4.0 Control Systems with MOM-based Hybrid Architecture. In *2024 the 12th International Conference on Information Technology (ICIT) (ICIT 2024), December 13–15, 2024, Kuala Lumpur, Malaysia.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3718391.3727888

## 1 Introduction

The design and implementation of software architectures significantly influence the success of modern applications. Two prominent architectural paradigms—Monolithic Architecture (MA) and Microservice-based Architecture (MSA)—have emerged, each with unique advantages and limitations. MA integrates all application components and functionalities into a single, cohesive unit. This traditional approach is simple and efficient for basic applications. However, as complexity grows, it becomes unwieldy, difficult to manage, and lacks scalability. In contrast, MSA, inspired by agile principles, breaks applications into small, independent, and loosely coupled services [1]. Each service focuses on a specific functionality, enabling scalability, flexibility, and easier management of complex systems. Leading organizations like Google, Netflix, Amazon, and eBay [2] have transitioned from MA to MSA to enhance agility, scalability, and fault tolerance in their dynamic, customer-centric applications. MSA offers numerous advantages recognized by both academic and industry circles, including easier maintenance, code reusability, scalable infrastructure, high availability, Continuous Integration/Continuous Deployment (CI/CD) processes, and Independent/Decoupled Development (ID/DD). According to Martin Fowler, MSA enables building applications by breaking them into small, independent services that communicate using lightweight protocols. This paradigm emphasizes nine key features, such as

service organization, decentralized decision-making, and data management I decentralized environment [3]. It adheres to the Single Responsibility Principle (SRP), where each service performs one specific task [4]. MSA also supports polyglot microservices (PMS), allowing the use of multiple programming languages within the same application [5]. This research introduces a hybrid architecture concept applied to Supervisory Control and Data Acquisition (SCADA) systems. SCADA systems, critical for monitoring and automating production processes, have evolved with the advent of Industry 4.0. This industrial revolution promotes smart systems leveraging emerging Information and Communication Technologies (ICT) for various systems [6].

The proposed hybrid approach (HA) integrates a common shareable database among modules, internal communication via Message-Oriented Middleware (MOM) protocols like RabbitMQ, (Advanced Message Queuing Protocol) and MQTT (Message Queuing Telemetry Transport), and external communication through a REST API gateway. This architecture addresses challenges such as data consistency, modular development, scalability, and interoperability with legacy systems while maintaining robust security protocols. Incorporating MOM enhances scalability, fault isolation, and resource efficiency. However, further optimization of hybrid architectures is necessary to tackle challenges such as resource efficiency and adaptability to modern demands.

The paper is organized as follows: Section II explores architecture patterns and migration reasons, Section III presents the proposed approach, Section IV details a traffic management case study, Section V analyzes results, and Section VI concludes with future directions for enhancing hybrid architectures.

## 2 Pattern Analysis of MA & MSA

MA is known for its simplicity and centralized data management but struggles with scalability and resource inefficiencies. In contrast, MSA provides flexibility and scalability but introduces complexity, particularly in maintaining data consistency and communication between services [7]. The key differences between MA and MSA patterns revolve around the following constraints [8]:

- Constraint (1): Modularity and Independence
- Constraint (2): Deployment and Scalability
- Constraint (3): Fault Tolerance and Resilience
- Constraint (4): Development and Maintenance
- Constraint (5): Data Management

MA is easier to implement initially but becomes cumbersome as applications scale. Conversely, MSA offers better scalability, flexibility, and resilience, though it requires complex management and coordination [9]. These differences highlight the trade-offs between MA and MSA, as depicted in Figure 1, emphasizing the importance of selecting the right architecture for specific use cases.

### 2.1 Reasons of Migration

MSA is rapidly gaining traction in the software industry, with many companies transitioning from traditional MA and Service-Oriented Architecture (SOA) to MSA due to its numerous advantages [10]. In a monolithic application, all components and functions are tightly integrated into a single, cohesive unit, including the user interface, business logic, and data access layers, all within one codebase and deployment [11]. In contrast, a microservice-based application is a collection of small, independently deployable services, each focusing on a specific business functionality and communicating via lightweight protocols like HTTP/REST or MOM [12]. Both architectures presented in Figure 2 and Figure 3 respectively.

As digital services grow, scalability, maintainability, performance, and security become crucial. Scaling a monolithic architecture is costly and limited to the entire application, making it less adaptable to modern demands [13]. MSA allows more flexible and cost-effective scaling of individual services, addressing this challenge [14]. Monolithic systems are tightly coupled, so changes in one part can affect the whole system, complicating maintenance. In contrast, microservices operate independently, facilitating easier updates, feature additions, high testability, and efficient CI/CD pipelines for continuous integration and deployment [15]. These benefits, including scalability, flexibility, and easier deployment, drive the industry's shift toward MSA [16]. The migration from monolithic systems to microservices typically follows three principles: refactoring existing code incrementally, rebuilding the application with new technologies, or creating a new system from scratch [17][18].

## 3 Proposed Hybrid Architecture

This paper focuses on Hybrid Architecture (HA), which combines the strengths of both Monolithic Architecture (MA) and Microservice-based Architecture (MSA). In Industry 4.0 applications, such as manufacturing, energy management, supply chain logistics, and process control, hybrid architecture provides a flexible solution. It typically uses a stable monolithic core for fundamental control and real-time operations while incorporating microservices for scalable, distributed tasks like data analysis, predictive maintenance, remote monitoring, and analytics. For example, in Manufacturing Execution Systems (MES), a hybrid model centralizes core functions like production scheduling for stability, while allowing scalability and flexibility for modules like data analysis or quality control. This approach leverages a shared database to ensure data consistency and simplify management. Master Data Management (MDM) tools are used to resolve data inconsistencies. The hybrid architecture also incorporates fault tolerance and fast request processing to ensure 100% system availability, providing optimal performance and efficiency in complex industrial systems.

### 3.1 HA Core Characteristics

This paper explores the advantages of hybrid architecture, focusing on data consistency, modular development, and enhanced scalability. The approach is applied in various industries such as MES, EMS, SCADA, Automated Industrial Equipment and Health & Safety Monitoring system which often require a balance between centralized control and distributed, scalable data handling. For SCADA systems, hybrid architecture is designed to meet diverse needs, including real-time control, scalability, and data consistency. Its key features are:

1. **Distributed**: Components or services are distributed across different nodes or servers, promoting resource utilization and load balancing.

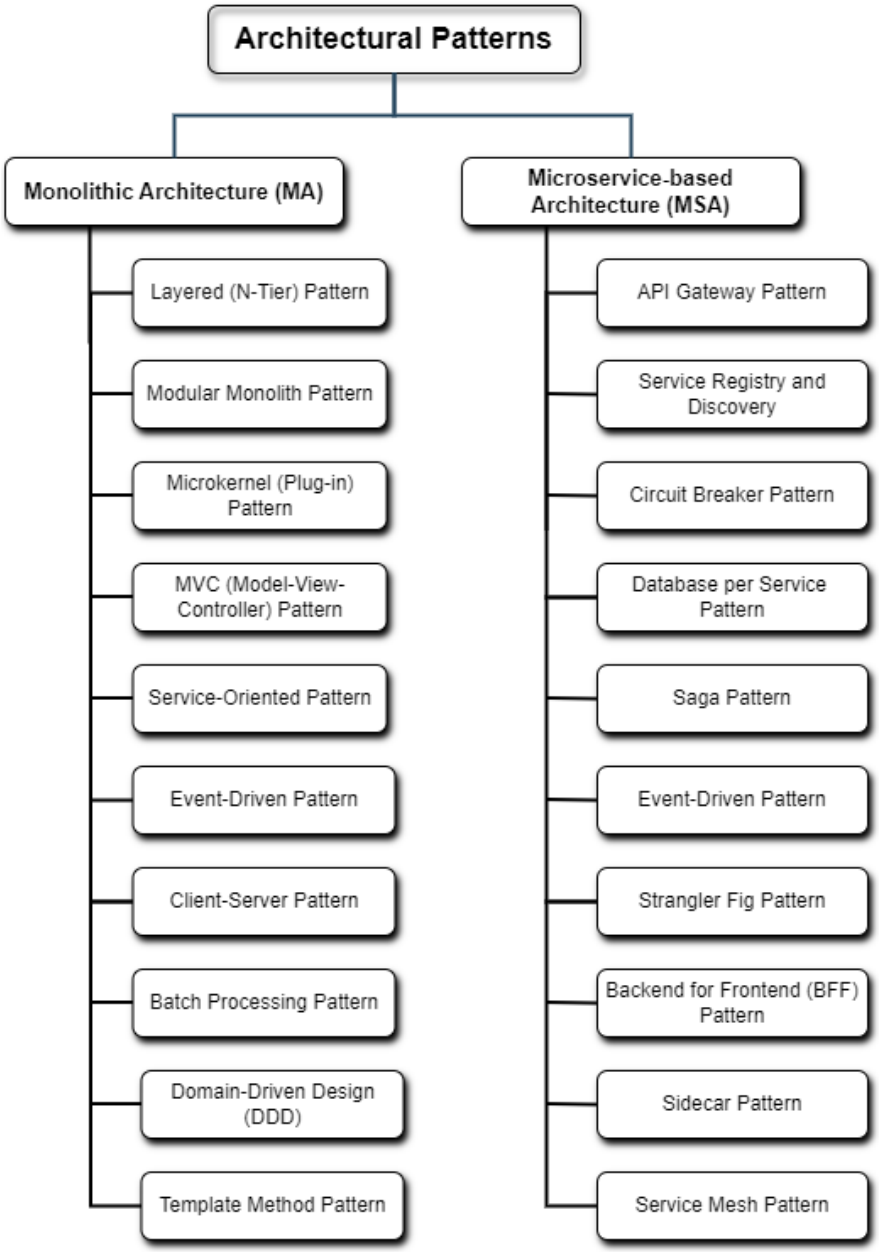



**Figure 1: Architecture Patterns: MA v/s MSA**

2. **Real-Time Capabilities**: Handles real-time data processing, monitoring, and control, immediate responses to events.
3. **Scalability**: Supports horizontal and vertical scaling to handle varying workloads and resource demands.
4. **Common Database**: Ensures data consistency and serves as a central repository for structured data.
5. **Modularity**: Facilitates independent development and deployment of components, promoting agility and code reusability.

## 3.2 System Architecture Design

Hybrid Architecture (HA) combines the strengths of both architectures to offer a balanced solution. The monolithic core plays a vital role in maintaining stability, control, and data consistency, while also enabling the flexibility and scalability that microservices provide. This dual approach caters to applications that require centralized control alongside distributed processing capabilities.

1. **Monolithic Core:** The Monolithic Core serves as the central controller, handling real-time operations, monitoring,

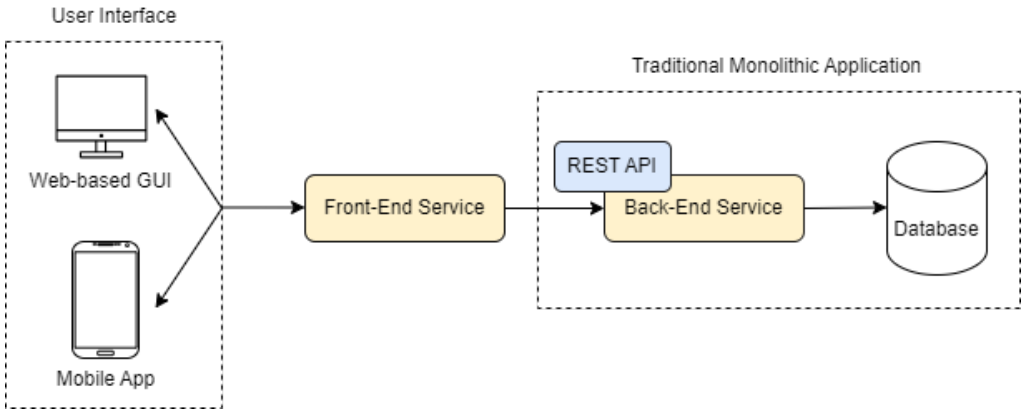



**Figure 2: Scratch Monolithic Application**

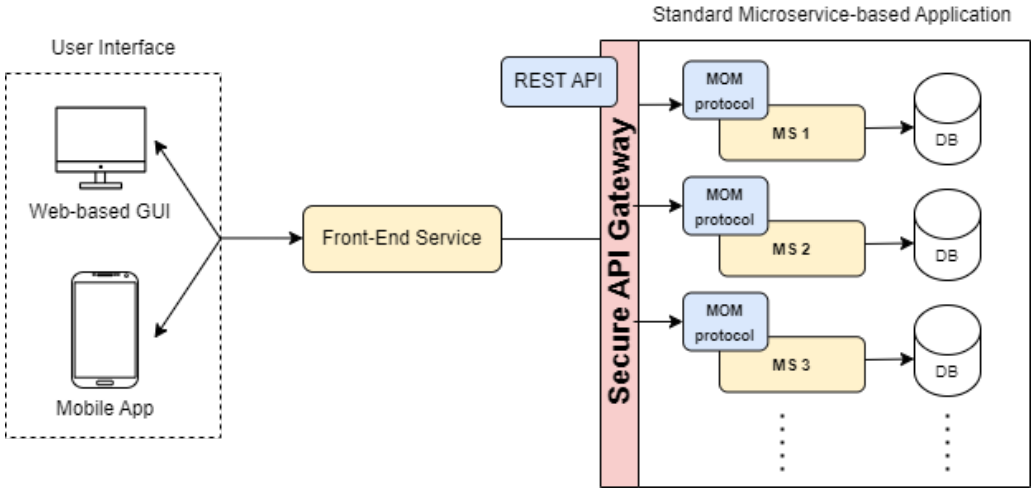



**Figure 3: Scratch Microservice-based Application**

and critical SCADA functions. This ensures the system's stability, responsiveness, and reliability, which is essential for industrial environments.

2. **Microservices Modules:** The system consists of various independent services, each focusing on specific tasks like data analysis, staff management, anomaly detection, and data collection. These services can be scaled individually to manage varying workloads.
3. **Common Database:** A shared database is deployed to store structured data and maintain consistency across both the monolithic core and microservices. This ensures that all components share and modify information from a unified data source.
4. **Gateway Module:** This is the core heart of the HA. It acts as a bridge between the public interfaces (mobile and web GUIs) and the internal microservices. It facilitates protocol translation, request routing, and security enforcement, ensuring only authorized users can access the system.
5. **Mobile and Web GUI:** The system offers **Mobile and Web GUIs** for real-time data visualization, user control, and access to system functionalities. These interfaces are essential for monitoring and interacting with the system from various devices, improving operational efficiency.

## 3.3 Key Performance Indicators of Hybrid Architecture

Measuring hybrid architecture performance involves using Key Performance Indicators (KPIs) to assess system efficiency, balance between the monolithic core and microservices, and ensure the architecture meets its objectives through various performance benchmarks.

**Response Time:**

- The duration required to process a user request from receipt to sending back a response.
- Significance: Indicates how quickly the system responds to user interactions.

- Measurement: Typically measured in milliseconds (ms) or seconds (s).

**Latency**

- The delay between a user action and the corresponding response from the system.
- Significance: Critical for real-time applications where delays can impact user experience.
- Measurement: Measured in milliseconds (ms).

**Throughput**

- The number of requests the system can process within a given time frame.
- Significance: Indicates the capacity of the system to handle high loads.
- Measurement: Measured in requests per second (RPS) or transactions per second (TPS).

**Scalability**

- The ability of the system to handle increased load by adding resources.
- Significance: Determines the system's capacity to grow and handle more requests.
- Measurement: Evaluating through load testing and monitoring resource scaling effects.

**CPU Utilization**

- The percentage of CPU resources used by the system to process requests.
- Significance: High CPU usage signals bottlenecks; low usage indicates underutilization.
- Measurement: Measured as a percentage (%).

**Memory Utilization**

- The amount of memory used by the system.
- Significance: Helps identify memory leaks or insufficient memory allocation.
- Measurement: Measured in megabytes (MB) or gigabytes (GB) and as a percentage (%).

**Average Error Rate**

- The percentage of failed requests compared to the total number of requests.
- Significance: High error rates can indicate issues with system stability and reliability.
- Measurement: Measured as a percentage (%).

**Availability**

- The proportion of time the system is operational and accessible.
- Significance: Essential for ensuring system reliability and user trust.
- Measurement: Measured as a percentage (%) uptime.

**Network Bandwidth**

- The amount of data transmitted over the network in a given time period.
- Significance: Affects the system's ability to handle large volumes of data transfer.
- Measurement: Measured in megabits per second (Mbps) or gigabits per second (Gbps).

**Fault Tolerance**

- A system's ability to function correctly despite failures or errors occurring.
- Significance: Ensures system reliability and availability, especially in environments where downtime or data loss is critical.
- Measurement: Measured in minutes or hours evaluated through the core metrics: Recovery Time Objective (RTO), Recovery Point Objective (RPO), Mean Time Between Failures (MTBF) and Mean Time to Repair (MTTR).

Regular monitoring, analyzing, and continuously optimizing these metrics is essential for maintaining the desired performance of a hybrid architecture. Performance testing and load testing should be performed regularly to identify bottlenecks and areas for improvement.

## 4 Case Study: Traffic Management System

This study focuses on comparing monolithic and microservice architectures by implementing and evaluating a hybrid architecture (HA) for a real-time Traffic Management System (TMS). The TMS is crucial for improving vehicle flow, reducing congestion, and enhancing urban safety. However, the monolithic architecture struggles with scalability and performance in handling the increasing complexity of traffic data. The hybrid architecture combines the simplicity of monolithic systems for core functionalities with the scalability and flexibility of microservices. The proposed architecture integrates system modules into a cohesive structure while leveraging **RabbitMQ** for asynchronous communication and reliable messaging. **SQLAlchemy** facilitates database interactions, ensuring efficient data management. The architecture employs an **API gateway** for client requests, managing routing, security, protocol translation, load balancing, and caching. Communication between public interface modules and gateway uses **RESTful HTTP requests**, while API gateway translates them into internal messaging protocols (RabbitMQ). To further enhance performance, scalability, and manageability, **Zuul** and **Nginx** are integrated. Zuul, a **JVM-based API gateway**, manages dynamic routing, monitoring, and security configurations, ensuring resilience and scalability. Nginx serves as the entry point for all client requests, handling SSL/TLS termination, static content delivery, caching, and initial load balancing.

The hierarchical workflow of TMS involves **NGINX, Zuul API gateway, databases, and Eureka-based service layers**, each with its own **load balancer (LB)** to improve system performance, presented in Figure 4. The proposed architecture for TMS, aims to provide a robust solution by combining monolithic and microservice approaches, explained in Figure 5. Key performance metrics, including hardware-based (CPU and Memory utilization), throughput, and average error rate, are used to evaluate system responsiveness and resource efficiency. These metrics are crucial for comparing the performance of the hybrid system against existing approaches. This research presents a scalable and efficient hybrid architecture, combining the strengths of MA and MSA, tailored to meet the demands of complex, real-time TMS.

## 5 Results and Discussion

In this section, we leverage 3 critical performance metrics to evaluate and contrast the three architecture design patterns which are

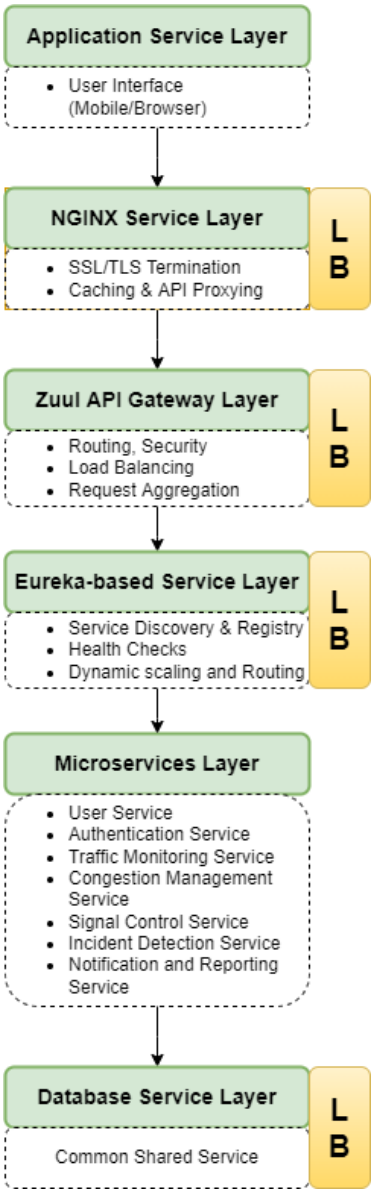



**Figure 4: Traffic Management System: Tiered Workflow Structure**

Traditional MA, Eureka-Zuul MSA and Proposed HA. These assessments were conducted on an operational traffic management system, providing practical insights into their performance. The metrics scrutinized in this analysis are evaluated using the following methodologies:

CPU Utilization: This metric indicates how much of the host system's processing power is being used. By monitoring the CPU usage of the system, we can assess its efficiency. The data presented in Table 1 reveals that CPU usage for the proposed hybrid architecture application is lower compared to the other two approaches.

**Table 1: CPU utilization comparison across architectures**

| Approach/Algorithm | Avg. CPU Utilization (%) |
|---|---|
| Traditional MA | 54.6 |
| Eureka-Zuul MSA | 46.8 |
| Proposed HA | 42.8 |

Consequently, the memory usage is also expected to be lower for the proposed approach.

Throughput: Upon analyzing the data, it is found that the proposed architecture consistently exhibits higher throughput values compared to other two approaches across all user levels and performs better in handling increased user loads and is more efficient in processing requests or transactions presented in Table 2. For implementing this comparison for throughput, Apache JMeter tool is utilized for load testing. It provides a feature to simulate multiple users accessing the system concurrently, leading with the necessary metrics to evaluate throughput.

Average Error Rate: It quantifies the accuracy of a system's responses or outputs compared to the expected or desired outcomes. In the context of TMS, average error rate measures the frequency of discrepancies or inaccuracies in the system's operations, such as traffic monitoring, signal control, incident detection, and other functionalities presented in Table 3. A lower average error rate indicates higher accuracy and reliability of the system in performing its tasks.

## 6 Conclusion and Future work

This paper explored the architectural paradigms of monolithic, microservice, and hybrid systems, specifically focusing on their application within a SCADA system for traffic management. However, the transition to microservices introduces complexities in data management, service coordination, and increased operational overhead. A hybrid architecture leverages the strengths of both paradigms, combining the simplicity and directness of monolithic design with the scalability and fault tolerance of microservices. In our SCADA system case study, the hybrid architecture demonstrated improved performance metrics such as reduced CPU utilization, better throughput, and lower error rates compared to purely monolithic or microservice-based systems. This design not only enhances the system's scalability and resilience but also ensures efficient handling of traffic data, making it well-suited for modern industry 4.0 applications. Future research should focus on optimizing the hybrid architecture for different industrial applications and exploring automated tools for seamless migration from monolithic to hybrid systems. Integration of adaptive framework for enhancing scalability with Kubernetes, improving fault tolerance, and integrating advanced security measures. Additionally, implementing real-time data processing and leveraging cloud services for better resource management will further optimize the system.

## References

[1] R. Chen, S. Li and Z. Li, "From Monolith to Microservices: A Dataflow-Driven Approach," 2017 24th Asia-Pacific Software Engineering Conference (APSEC), Nanjing, China, 2017, pp. 466-475, doi: 10.1109/APSEC.2017.53.

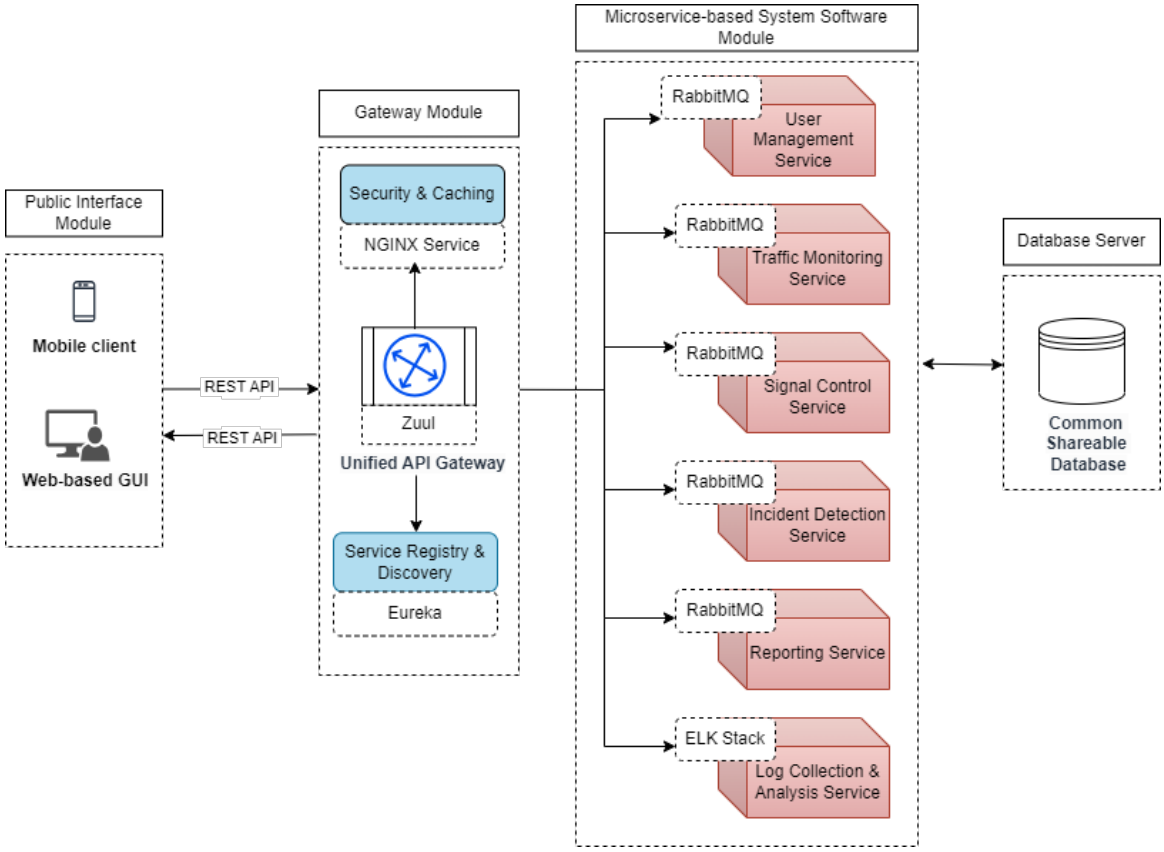



**Figure 5: Hybrid Architecture: Traffic Management System**

**Table 2: Throughput (success per seconds) comparison across architectures**

| No. of Users | Traditional MA | Eureka-Zuul MSA | Proposed HA |
|---|---|---|---|
| 1 | 22 | 26 | 28 |
| 10 | 34 | 37 | 40 |
| 20 | 46 | 48 | 51 |
| 40 | 65 | 70 | 73 |
| 60 | 78 | 80 | 85 |

**Table 3: Average error rate comparison across architectures**

| No. of Users | Traditional MA | Eureka-Zuul MSA | Proposed HA |
|---|---|---|---|
| 1 | 2 | 1 | 1 |
| 10 | 10 | 8 | 7 |
| 50 | 13 | 12 | 10 |
| 100 | 15 | 13 | 12 |
| 200 | 18 | 15 | 13 |

[2] P. D. Francesco, I. Malavolta and P. Lago, "Research on Architecting Microservices: Trends, Focus, and Potential for Industrial Adoption," 2017 IEEE International Conference on Software Architecture (ICSA), Gothenburg, Sweden, 2017, pp. 21-30, doi: 10.1109/ICSA.2017.24.

[3] M. Fowler and J. Lewis, "Microservices," 10 June 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html

[4] M. Kalske, N. Mäkitalo, and T. Mikkonen, 'Challenges When Moving from Monolith to Microservice Architecture', in ICWE Workshops, 2017.

[5] Kazanavičius, Justas, Dalius Mažeika, and Diana Kalibatienė. 2022. "An Approach to Migrate a Monolith Database into Multi-Model Polyglot Persistence Based on Microservice Architecture: A Case Study for Mainframe Database" Applied Sciences 12, no. 12: 6189. https://doi.org/10.3390/app12126189

[6] R. Rastogi, A. Arya, A. Chola, A. Chaudhary and P. Kaur, "Intelligent Maintenance Scheduling and Decision Support Model for Industry 4.0 System Based on Severity Factors and Priority Weights," 2024 Third International Conference on Power, Control and Computing Technologies (ICPC2T), Raipur, India, 2024, pp. 523-528, doi: 10.1109/ICPC2T60072.2024.10475018.

[7] N. Bjørndal, A. Bucchiarone, M. Mazzara, N. Dragoni, S. Dustdar, F. B. Kessler, and T. Wien, "Migration from monolith to microservices: Benchmarking a case study", Tech. Rep., 2020.

[8] G. Márquez and H. Astudillo, "Actual Use of Architectural Patterns in Microservices-Based Open Source Projects," 2018 25th Asia-Pacific Software Engineering Conference (APSEC), Nara, Japan, 2018, pp. 31-40, doi: 10.1109/APSEC.2018.00017.

[9] S. Vergara, L. González and R. Ruggia, "Towards Formalizing Microservices Architectural Patterns with Event-B," 2020 IEEE International Conference on Software Architecture Companion (ICSA-C), Salvador, Brazil, 2020, pp. 71-74, doi: 10.1109/ICSA-C50368.2020.00022.

[10] Sidath Weerasinghe, Indika Perera, "Taxonomical Classification and Systematic Review on Microservices," International Journal of Engineering Trends and Technology, vol. 70, no. 3, pp. 222-233, 2022. Crossref, https://doi.org/10.14445/22315381/IJETT-V70I3P225

[11] D. Kuryazov, D. Jabborov and B. Khujamuratov, "Towards Decomposing Monolithic Applications into Microservices," 2020 IEEE 14th International Conference on Application of Information and Communication Technologies (AICT), Tashkent, Uzbekistan, 2020, pp. 1-4, doi: 10.1109/AICT50176.2020.9368571.

[12] O. Al-Debagy and P. Martinek, "Extracting Microservices' Candidates from Monolithic Applications: Interface Analysis and Evaluation Metrics Approach," 2020 IEEE 15th International Conference of System of Systems Engineering (SoSE), Budapest, Hungary, 2020, pp. 289-294, doi: 10.1109/SoSE50414.2020.9130466.

[13] Y. Romani, O. Tibermacine and C. Tibermacine, "Towards Migrating Legacy Software Systems to Microservice-based Architectures: a Data-Centric Process for Microservice Identification," 2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C), Honolulu, HI, USA, 2022, pp. 15-19, doi: 10.1109/ICSA-C54293.2022.00010.

[14] L. D. S. B. Weerasinghe and I. Perera, "An exploratory evaluation of replacing ESB with microservices in service-oriented architecture," 2021 International Research Conference on Smart Computing and Systems Engineering (SCSE), Colombo, Sri Lanka, 2021, pp. 137-144, doi: 10.1109/SCSE53661.2021.9568289.

[15] K. Munonye and P. Martinek, "Evaluation of Data Storage Patterns in Microservices Archicture," 2020 IEEE 15th International Conference of System of Systems Engineering (SoSE), Budapest, Hungary, 2020, pp. 373-380, doi: 10.1109/SoSE50414.2020.9130516.

[16] T. Cerny *et al.*, "On Code Analysis Opportunities and Challenges for Enterprise Systems and Microservices," in IEEE Access, vol. 8, pp. 159449-159470, 2020, doi: 10.1109/ACCESS.2020.3019985.

[17] N. C. Mendonca, P. Jamshidi, D. Garlan and C. Pahl, "Developing Self-Adaptive Microservice Systems: Challenges and Directions," in IEEE Software, vol. 38, no. 2, pp. 70-79, March-April 2021, doi: 10.1109/MS.2019.2955937.

[18] F. Ponce, G. Márquez and H. Astudillo, "Migrating from monolithic architecture to microservices: A Rapid Review," 2019 38th International Conference of the Chilean Computer Science Society (SCCC), Concepcion, Chile, 2019, pp. 1-7, doi: 10.1109/SCCC49216.2019.8966423.